JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class PushbackReader

java.lang.Object
  java.io.Reader
    java.io.FilterReader
      java.io.PushbackReader

All Implemented Interfaces:

Closeable, AutoCloseable, Readable

```
public class PushbackReader
extends FilterReader
```

A character-stream reader that allows characters to be pushed back into the stream.

Since:

1.1

Field Summary

Fields declared in class FilterReader

in

Fields declared in class Reader

lock

Constructor Summary

Constructors

Constructor	Description
PushbackReader(Reader in)	Creates a new pushback reader with a one-character pushback buffer.
PushbackReader(Reader in, int size)	Creates a new pushback reader with a pushback buffer of the given size.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description

void	close()	Closes the stream and releases any system resources associated with it.
void	mark (int readAheadLimit)	Marks the present position in the stream.
boolean	markSupported()	Tells whether this stream supports the mark() operation, which it does not.
int	read()	Reads a single character.
boolean	ready()	Tells whether this stream is ready to be read.
void	reset()	Resets the stream.
void	unread(char[] cbuf)	Pushes back an array of characters by copying it to the front of the pushback buffer.
void	unread(char[] cbuf, int off, int len)	Pushes back a portion of an array of characters by copying it to the front of the pushback buffer.
void	unread(int c)	Pushes back a single character by copying it to the front of the pushback buffer.

Methods declared in class FilterReader

read, skip

Methods declared in class Reader

nullReader, of, read, read, readAllAsString, readAllLines, transferTo

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

PushbackReader

```
public PushbackReader(Reader in,
                      int size)
```

Creates a new pushback reader with a pushback buffer of the given size.

Parameters:

in - The reader from which characters will be read

size - The size of the pushback buffer

Throws:

IllegalArgumentException - if size <= 0

PushbackReader

```
public PushbackReader(Reader in)
```

Creates a new pushback reader with a one-character pushback buffer.

Parameters:

in - The reader from which characters will be read

Method Details

read

```
public int read()
         throws IOException
```

Reads a single character.

Overrides:

read in class FilterReader

Returns:

The character read, or -1 if the end of the stream has been reached

Throws:

IOException - If an I/O error occurs

unread

```
public void unread(int c)
            throws IOException
```

Pushes back a single character by copying it to the front of the pushback buffer. After this method returns, the next character to be read will have the value `(char)c`.

Parameters:

`c` - The int value representing a character to be pushed back

Throws:

`IOException` - If the pushback buffer is full, or if some other I/O error occurs

unread

```
public void unread(char[] cbuf,
                   int off,
                   int len)
            throws IOException
```

Pushes back a portion of an array of characters by copying it to the front of the pushback buffer. After this method returns, the next character to be read will have the value `cbuf[off]`, the character after that will have the value `cbuf[off+1]`, and so forth.

Parameters:

`cbuf` - Character array

`off` - Offset of first character to push back

`len` - Number of characters to push back

Throws:

`IOException` - If there is insufficient room in the pushback buffer, or if some other I/O error occurs

unread

```
public void unread(char[] cbuf)
            throws IOException
```

Pushes back an array of characters by copying it to the front of the pushback buffer. After this method returns, the next character to be read will have the value `cbuf[0]`, the character after that will have the value `cbuf[1]`, and so forth.

Parameters:

`cbuf` - Character array to push back

Throws:

`IOException` - If there is insufficient room in the pushback buffer, or if some other I/O error occurs

ready

```
public boolean ready()
              throws IOException
```

Tells whether this stream is ready to be read.

Overrides:

`ready` in class `FilterReader`

Returns:

True if the next read() is guaranteed not to block for input, false otherwise. Note that returning false does not guarantee that the next read will block.

Throws:

`IOException` - If an I/O error occurs

mark

```
public void mark(int readAheadLimit)
          throws IOException
```

Marks the present position in the stream. The `mark` for class `PushbackReader` always throws an exception.

Overrides:

`mark` in class `FilterReader`

Parameters:

`readAheadLimit` - Limit on the number of characters that may be read while still preserving the mark. After reading this many characters, attempting to reset the stream may fail.

Throws:

`IOException` - Always, since mark is not supported

reset

```
public void reset()
           throws IOException
```

Resets the stream. The `reset` method of `PushbackReader` always throws an exception.

Overrides:

reset in class FilterReader

Throws:

IOException - Always, since reset is not supported

markSupported

```
public boolean markSupported()
```

Tells whether this stream supports the mark() operation, which it does not.

Overrides:

markSupported in class FilterReader

Returns:

true if and only if this stream supports the mark operation.

close

```
public void close()
           throws IOException
```

Closes the stream and releases any system resources associated with it. Once the stream has been closed, further read(), unread(), ready(), or skip() invocations will throw an IOException. Closing a previously closed stream has no effect. This method will block while there is another thread blocking on the reader.

Specified by:

close in interface AutoCloseable

Specified by:

close in interface Closeable

Specified by:

close in class Reader

Throws:

IOException - If an I/O error occurs

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.